

Secure Backend Engineering using Flask: Middleware Integration & RESTful API Development

1. Introduction

Modern web applications require a secure and scalable backend capable of:

- Handling client requests
- Managing user authentication
- Processing business logic
- Accessing databases
- Protecting sensitive information
- Exposing services through APIs

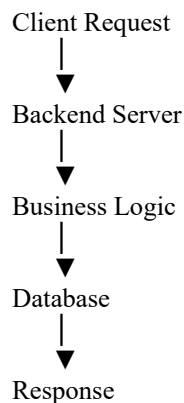
Flask is widely used for backend engineering because it is:

- Lightweight
- Flexible
- Easy to extend
- REST API friendly
- Suitable for microservices architecture

2. What is Backend Engineering?

Definition

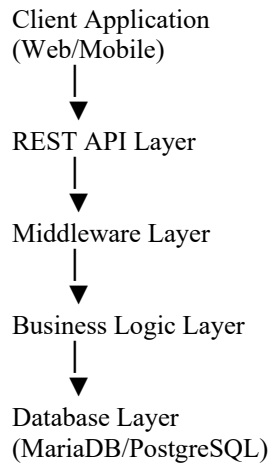
Backend Engineering refers to the development of server-side components responsible for:



The backend performs:

- Authentication
- Authorization
- Data Processing
- Database Operations
- API Management
- Security Enforcement

3. Backend Architecture in Flask



4. Secure Backend Engineering

Definition

Secure Backend Engineering involves implementing security controls that protect:

- Users
- APIs
- Databases
- Sessions
- Sensitive Data

against:

- SQL Injection
- Cross-Site Scripting (XSS)
- CSRF
- Session Hijacking
- Authentication Bypass
- Unauthorized Access

5. Security Principles

Authentication

Verifies identity.

Example:

Username
Password

Authorization

Determines permissions.

Example:

Admin
Faculty
Student

Confidentiality

Sensitive information remains private.

Integrity

Data remains unchanged and trustworthy.

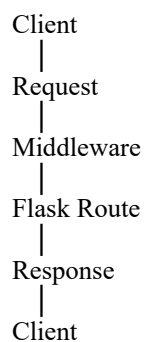
Availability

Services remain accessible.

6. What is Middleware?

Definition

Middleware is software that sits between the client request and application logic.

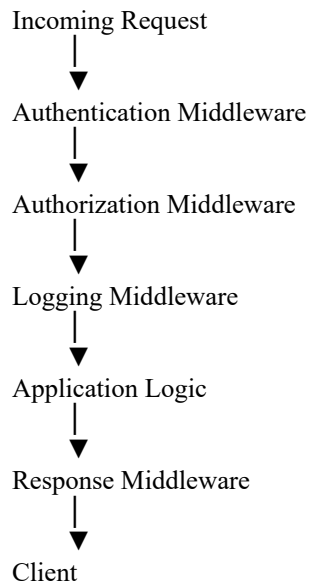


Middleware can:

- Authenticate users
- Log requests
- Validate tokens
- Rate limit requests

- Handle errors

7. Middleware Architecture



8. Middleware in Flask

Flask provides:

```
@app.before_request
```

and

```
@app.after_request
```

Example

```
from flask import Flask

app = Flask(__name__)

@app.before_request
def before():

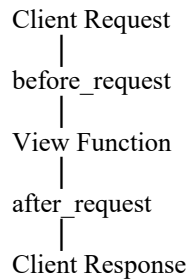
    print("Request Received")

@app.after_request
def after(response):

    print("Response Sent")
```

```
return response
```

Request Lifecycle



9. Authentication Middleware

Authentication middleware verifies user identity before processing requests.

Example:

```
from flask import request

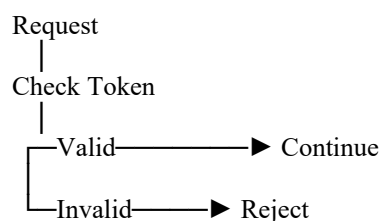
@app.before_request
def authenticate():

    token = request.headers.get(
        "Authorization"
    )

    if token != "secret":

        return {
            "error": "Unauthorized"
        }, 401
```

Workflow



10. Logging Middleware

Logs user activities.

```
@app.before_request
def logger():
```

```
    print(
        request.method,
        request.url
    )
```

Output:

```
GET /students
POST /login
```

11. Error Handling Middleware

```
@app.errorhandler(404)
def not_found(error):

    return {
        "message": "Page Not Found"
    }, 404
```

12. CORS Middleware

What is CORS?

Cross-Origin Resource Sharing allows frontend applications from different domains to access APIs.

Example:

```
Frontend
localhost:3000
```

```
Backend
localhost:5000
```

Without CORS:

```
Blocked
```

Install

```
pip install flask-cors
```

Configure

```
from flask_cors import CORS

CORS(app)
```

13. Security Headers Middleware

```
@app.after_request
def secure_headers(response):

    response.headers[
        "X-Frame-Options"
    ] = "DENY"

    response.headers[
        "X-Content-Type-Options"
    ] = "nosniff"

    return response
```

14. Rate Limiting Middleware

Protects APIs from abuse.

Install:

```
pip install flask-limiter
```

Example:

```
from flask_limiter import Limiter

limiter = Limiter(
    app=app,
    default_limits=["100/day"]
)
```

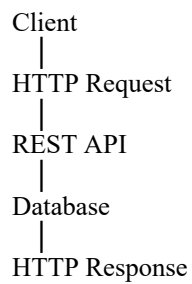
15. What is REST?

REST

Representational State Transfer

A REST API allows communication using HTTP protocols.

REST Architecture



REST Principles

Stateless

Each request contains complete information.

Example:

Request 1 Independent
Request 2 Independent

Client-Server

Frontend and backend remain separate.

Uniform Interface

Uses standard HTTP methods.

16. HTTP Methods

Method	Purpose
GET	Read Data
POST	Create Data
PUT	Update Data
DELETE	Delete Data
PATCH	Partial Update

Example

GET /students
POST /students
PUT /students/1
DELETE /students/1

17. Creating REST API in Flask

Basic API

```
from flask import Flask

app = Flask(__name__)

@app.route("/")

def home():

    return {
        "message": "Welcome"
    }
```

Output:

```
{
  "message": "Welcome"
}
```

18. GET API

```
@app.route(
    "/students",
    methods=["GET"]
)
def students():

    return {

        "students": [
            "John",
            "Alice"
        ]
    }
```

Output:

```
{
  "students":[
    "John",
    "Alice"
  ]
}
```

19. POST API

```
from flask import request

@app.route(
    "/student",
    methods=["POST"]
)

def add_student():

    data = request.json

    return {

        "message":"Added",

        "student":data

    }
```

Request

```
{
  "name":"John",
  "marks":85
}
```

Response

```
{
  "message":"Added",
  "student":{
    "name":"John",
    "marks":85
  }
}
```

20. PUT API

```
@app.route(
    "/student/<id>",
```

```

        methods=["PUT"]
    )

    def update_student(id):

        return {

            "message":
                f"Student {id} Updated"

        }

```

21. DELETE API

```

@app.route(
    "/student/<id>",
    methods=["DELETE"]
)

def delete_student(id):

    return {

        "message":
            f"Student {id} Deleted"

    }

```

22. Flask RESTful Extension

Install:

```
pip install flask-restful
```

Resource-Based API

```

from flask_restful import Resource

class Student(Resource):

    def get(self):

        return {
            "name": "John"
        }

```

API Registration

```
api.add_resource(  
    Student,  
    "/student"  
)
```

23. JSON Web Tokens (JWT)

Why JWT?

Secure API authentication.

Install:

```
pip install flask-jwt-extended
```

JWT Structure

```
Header  
.  
Payload  
.  
Signature
```

Example:

```
eyJhbGciOi...
```

Generate Token

```
from flask_jwt_extended import (  
    create_access_token  
)  
  
token = create_access_token(  
    identity="admin"  
)
```

Protect Route

```
from flask_jwt_extended import (  
    jwt_required  
)  
  
@app.route("/profile")
```

```
@jwt_required()

def profile():

    return {
        "message": "Secure"
    }
```

Secure API Flow

```
Login
|
JWT Generated
|
Client Stores Token
|
Protected API
|
Token Validation
|
Response
```

24. Flask + MariaDB REST API

Database Connection

```
import mariadb

conn = mariadb.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)
```

Retrieve Records

```
@app.route(
    "/students",
    methods=["GET"]
)

def students():

    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM students"
    )

    rows = cursor.fetchall()
```

```
return {  
    "students":rows  
}
```

25. API Security Best Practices

Use HTTPS

```
https://example.com
```

Validate Inputs

```
if marks < 0:  
  
    return {  
        "error":"Invalid Marks"  
    }
```

Parameterized Queries

Safe:

```
cursor.execute(  
    """  
    SELECT *  
    FROM students  
    WHERE id=?  
    """,  
    (id,) )
```

Store Passwords Securely

Use hashing:

```
from werkzeug.security import (  
    generate_password_hash  
)
```

Avoid Hardcoded Secrets

Bad:

```
SECRET_KEY="admin123"
```

Use:

```
import os

SECRET_KEY = os.getenv(
    "SECRET_KEY"
)
```

26. Complete Secure Backend Architecture

Frontend
(React/Angular)

|



REST API Gateway

|



Authentication Middleware

|



Authorization Middleware

|



Logging Middleware

|



Flask Business Logic

|



MariaDB Database

|



JSON Response

Real-World Applications

Secure Flask backends are widely used in:

- Student Management Systems
- Banking Applications
- Healthcare Systems
- ERP Platforms
- E-Commerce Applications
- Cyber Security Dashboards
- SIEM Solutions
- Digital Forensics Platforms
- IoT Management Systems
- Machine Learning APIs

Advantages of Flask for Secure Backend Engineering

1. Lightweight and flexible.
2. Easy middleware integration.
3. Strong REST API support.
4. JWT authentication support.
5. Database integration with MariaDB/PostgreSQL.
6. Scalable microservices architecture.
7. Extensive security extensions.

Summary

Secure Backend Engineering using Flask involves building robust server-side applications that enforce authentication, authorization, logging, input validation, and secure communication. Middleware acts as an intermediate processing layer that can inspect, modify, validate, or reject requests before they reach application logic. Flask's middleware capabilities (`before_request`, `after_request`) combined with extensions such as JWT authentication, CORS, rate limiting, and security headers provide a strong foundation for secure applications. RESTful API development in Flask enables standardized communication using HTTP methods (GET, POST, PUT, DELETE) and JSON data formats, making Flask a popular choice for modern web applications, cloud-native services, cybersecurity platforms, and enterprise backend systems.

Experiment 01: Write a step-by-step process to build a program to apply the "Secure Backend Engineering using Flask: Middleware Integration & RESTful API Development" on the Student Marks Management System.

Applying Secure Backend Engineering: Middleware Integration & RESTful API Development

This practical project extends the basic **Student Marks Management System** and transforms it into a secure, production-style Flask application with:

- RESTful APIs
- Middleware Integration
- Authentication
- Authorization
- Logging
- MariaDB Connectivity
- CSV Storage
- Security Controls

Phase 1: System Requirements Analysis

Functional Requirements

The system should:

1. Accept student information.
2. Accept marks for:
 - o English
 - o Hindi
 - o Maths
 - o SST
 - o Science
3. Calculate:
 - o Total
 - o Percentage
4. Store records in:
 - o CSV file
 - o MariaDB database
5. Retrieve student records.
6. Update student records.
7. Delete student records.

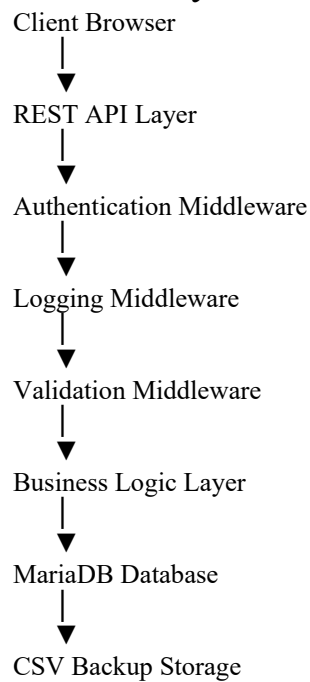
Security Requirements

The system should:

1. Authenticate users.
2. Authorize API access.

3. Log all requests.
4. Validate inputs.
5. Protect APIs using JWT.
6. Prevent SQL Injection.
7. Support CORS.
8. Handle errors securely.

Phase 2: System Architecture Design



Phase 3: Install Required Packages

Create Virtual Environment

```
python -m venv venv
```

Activate:

```
venv\Scripts\activate
```

Install dependencies:

```
pip install flask
pip install flask-cors
pip install flask-jwt-extended
pip install pandas
pip install mariadb
pip install flask-limiter
```

Phase 4: Create Project Structure

StudentManagementSystem

- app.py
- config.py
- middleware.py
- database.py
- models.py
- routes.py
- data
 - student_marks.csv
- logs
 - app.log
- templates
 - index.html
 - dashboard.html

Phase 5: Configure MariaDB

Create Database:

```
CREATE DATABASE studentdb;
```

Use Database:

```
USE studentdb;
```

Create Table:

```
CREATE TABLE student_marks
(
  id INT AUTO_INCREMENT PRIMARY KEY,

  student_name VARCHAR(100),

  english INT,

  hindi INT,

  maths INT,

  sst INT,

  science INT,

  total INT,
```

```
percentage DECIMAL(5,2)
);
```

Phase 6: Database Connection Layer

database.py

```
import mariadb

def get_connection():

    return mariadb.connect(

        host="localhost",

        user="root",

        password="password",

        database="studentdb"

    )
```

Purpose:

- Centralized database management.
- Reusable connection logic.

Phase 7: Configure JWT Authentication

app.py

```
from flask_jwt_extended import JWTManager

app.config["JWT_SECRET_KEY"] = "supersecretkey"

jwt = JWTManager(app)
```

Purpose:

- Generate secure access tokens.
- Protect REST APIs.

Phase 8: Create Login API

```
from flask_jwt_extended import create_access_token

@app.route(
    "/login",
    methods=["POST"]
)

def login():

    username = request.json["username"]

    password = request.json["password"]

    if username == "admin" and password == "admin123":

        token = create_access_token(
            identity=username
        )

        return {
            "access_token":token
        }

    return {
        "message":"Invalid Credentials"
    },401
```

Phase 9: Create Authentication Middleware

Purpose:

Protect all student APIs.

```
from flask_jwt_extended import jwt_required

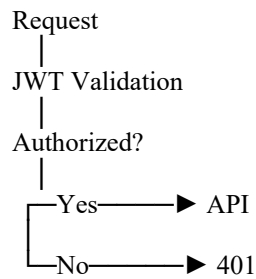
@app.route(
    "/students",
    methods=["GET"]
)

@jwt_required()

def students():

    pass
```

Workflow:



Phase 10: Create Logging Middleware

middleware.py

```
from flask import request

def log_request():

    print(
        request.method,
        request.url
    )
```

Register:

```
@app.before_request
def before():

    log_request()
```

Purpose:

- Audit trails
- User activity monitoring
- Cyber security monitoring

Phase 11: Add Security Headers Middleware

```
@app.after_request
def secure_headers(response):

    response.headers[
        "X-Frame-Options"
    ] = "DENY"

    response.headers[
        "X-Content-Type-Options"
    ] = "nosniff"
```

```
return response
```

Purpose:

- Prevent clickjacking
- Prevent MIME attacks

Phase 12: Enable CORS Middleware

```
from flask_cors import CORS
```

```
CORS(app)
```

Purpose:

Allow frontend applications to access APIs.

Example:

React Frontend
↓
Flask Backend

Phase 13: Input Validation Middleware

Validate student marks before processing.

```
def validate_marks(marks):  
  
    if marks < 0 or marks > 100:  
  
        return False  
  
    return True
```

Example:

```
if not validate_marks(english):  
  
    return {  
        "error": "Invalid Marks"  
    }
```

Purpose:

Prevent malicious input.

Phase 14: Create RESTful APIs

API 1: Add Student

POST /student

Request:

```
{
  "student_name": "John",
  "english": 85,
  "hindi": 90,
  "maths": 95,
  "sst": 88,
  "science": 92
}
```

Processing

total = english + hindi + maths + sst + science

percentage = total / 5

Store Data

1. CSV
2. MariaDB

Response

```
{
  "message": "Student Added"
}
```

API 2: Get All Students

GET /students

Purpose:

Retrieve all student records.

API 3: Get Student By ID

GET /student/1

Purpose:

Retrieve specific student.

API 4: Update Student

```
PUT /student/1
```

Purpose:

Modify marks.

API 5: Delete Student

```
DELETE /student/1
```

Purpose:

Remove student record.

Phase 15: Store Data in CSV

```
df.to_csv(  
  
    "student_marks.csv",  
  
    mode="a",  
  
    header=False,  
  
    index=False  
)
```

Purpose:

- Backup storage
- Reporting
- Data export

Phase 16: Store Data in MariaDB**Use parameterized queries:**

```

cursor.execute(

    """
    INSERT INTO student_marks
    VALUES
    (
        NULL,
        ?, ?, ?, ?, ?, ?, ?, ?
    )
    """,

    (

        student_name,

        english,

        hindi,

        maths,

        sst,

        science,

        total,

        percentage

    )

)

```

Purpose:

Prevent SQL Injection.

Phase 17: Error Handling

```

@app.errorhandler(404)

def not_found(error):

    return {

        "message": "Not Found"

    }, 404

```

Example:

```
try:

    conn.commit()

except Exception as e:

    return {

        "error":str(e)

    }
```

Phase 18: Rate Limiting

Install:

```
pip install flask-limiter
```

Configure:

```
from flask_limiter import Limiter

limiter = Limiter(
    app=app,
    default_limits=["100/day"]
)
```

Purpose:

Prevent API abuse.

Phase 19: Test APIs

Using:

- Postman
- Thunder Client
- Curl

Example:

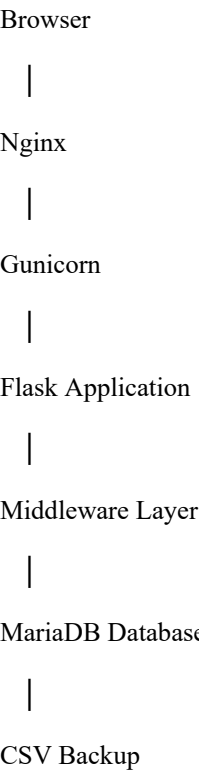
```
curl -X GET \
http://localhost:5000/students
```

Phase 20: Production Deployment

Use:

- Gunicorn
- Nginx
- Docker
- MariaDB

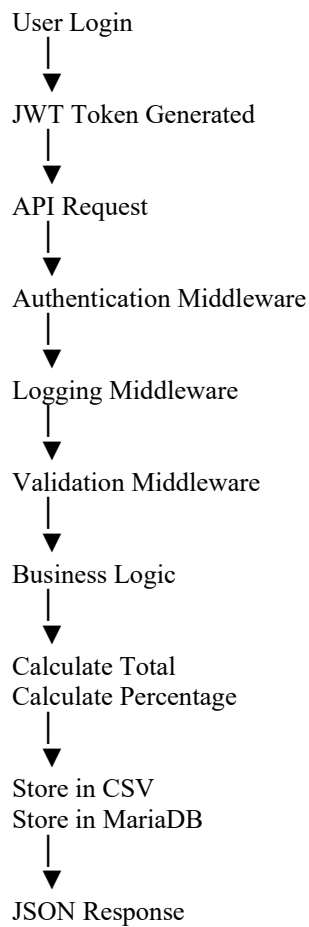
Architecture:



Security Features Implemented

Security Control	Purpose
JWT Authentication	Secure API Access
Authorization Middleware	User Validation
Logging Middleware	Audit Trails
CORS Middleware	Cross-Origin Access
Security Headers	Browser Protection
Input Validation	Data Integrity
Parameterized Queries	SQL Injection Prevention
Error Handling	Secure Failures
Rate Limiting	DoS Protection

Final Workflow



This implementation converts the basic Student Marks Management System into a **secure enterprise-style Flask backend** demonstrating:

- Flask Web Development
- RESTful API Design
- Middleware Integration
- JWT Authentication
- MariaDB Connectivity
- CSV Data Persistence